

NIYATI BHANANI

EPIC RPG DOC (1)

 MCA 4 Projects - 2025-26

Document Details

Submission ID**trn:oid::3618:128360498****Submission Date****Feb 16, 2026, 9:39 AM GMT+5:30****Download Date****Feb 16, 2026, 9:40 AM GMT+5:30****File Name****EPIC RPG DOC (1).docx****File Size****3.4 MB****20 Pages****2,173 Words****11,674 Characters**

55% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups



38 AI-generated only 55%

Likely AI-generated text from a large-language model.



0 AI-generated text that was AI-paraphrased 0%

Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



1.

1. SYNOPSIS:

- The Text-Based Python RPG Game is a command-line application developed in Python, allowing users to register, log in, create a character, move around different regions, fight off enemies, collect items, and gain experience points as the game progresses.
- The game is designed in such a way as to be very simple and easy to understand, ensuring that both playing the game and understanding the construction of the game are enjoyable and fun.
- The project has a modular design, with separate modules for user management, game functionality, inventory management, store functionality, and utility functions. This design makes the code easy to understand and maintain.
- The player's data, such as username, password, character attributes, experience points, in-game money, and inventory, are stored in JSON files, making it easy to save progress and pick up right where the player left off.
- The main point of this project is to show that fun and enjoyable gaming experiences can be created without the need for complex graphics or advanced development tools. The project shows that a complete and fun game can be built using simple logic and basic concepts in Python.
- The focus of this project is not only on the final product but also on making the process of game development understandable and enjoyable.
- Since it is a text-based application, the application can run on any system that has Python installed and does not require any special hardware. This project shows how basic programming concepts such as functions, loops, conditional statements, file handling, and modular programming can be used to create a real-world application.
- Overall, this project helps to improve the skills of logical reasoning, creativity, and confidence in game and application development using a simple yet effective approach.

2. PREAMBLE

2.1 General Introduction:

- Text-based RPGs (Role-Playing Games) are one of the earliest forms of computer games, built entirely through text prompts and user choices. These games are simple yet highly interactive, relying on decision-making rather than graphics.

2.2 Statement of Problem:

- Many beginner programmers find it difficult to understand how real games manage player progress, combat, and rewards. Most examples available are either too simple to be engaging or too complex to understand. This project aims to solve that problem by creating a simple, text-based RPG using basic Python, where core game mechanics like combat, crafting, inventory, and progression are easy to follow, modify, and learn from

2.3 Objective and Scope of the study:

- Implement user registration and login functionality.
- Created a system for character level progression and attribute management (health, XP, coins).
- Design and developed various game modules, including hunting, adventure, inventory management, a store, and character profiles.
- Incorporate file-based storage for user data persistence.
- Provide a user-friendly text-based interface with clear instructions and feedback.
- **Scope:**
- The project will include core RPG elements such as character development, combat, and inventory.
- The game will be text-based, focusing on gameplay mechanics rather than graphical presentation.
- User data will be stored in a JSON file.
- The game will be developed using Python 3.x.

2.4 Module Description with functionality:

2.4.1 User Management Module:

- Description: This module handles user registration, login, and deletion.
- Functions:
- register(): Creates a new user account, storing the username and password.
- login(): Authenticates existing users and retrieves their data.
- delete_user(): Removes a user account from the system.
- load_user_data(): Loads user data from the JSON file.
- save_user_data(): Saves user data to the JSON file.

Epic RPG

2.4.2 Game Mechanics Module:

- Description: This module contains the core gameplay functions, including hunting,
- adventure, chopping, and reward systems.
- Functions:
- `rpg_hunt()`: Simulates a hunting expedition where the player encounters creatures, gains
- rewards, and loses health.
- `rpg_adventure()`: Presents a more challenging adventure scenario with a single, powerful
- creature.
- `rpg_chop()`: Allows the player to chop wood and gather resources.
- `reward()`: Provides random rewards to the player.
- `get_level()`: Calculates the player's level based on their XP.
- `check_level_up()`: Checks if the player has leveled up and updates their stats

2.4.3 Inventory Module:

- Description: Manages the player's items.
- Function:
- `view_inventory()`: Displays the items currently in the player's inventory.

2.4.4 Profile Module:

- Description: Displays the player's character information.
- Function:
- `view_profile()`: Shows the player's username, level, stats (attack, defense, health), coins, and XP.

2.4.5 Store Module:

- Description: Allows the player to buy items from a store.
- Function:
- `store()`: Displays the store's inventory and handles purchases.

2.4.6 Main Game Module:

- Description: The main entry point of the game, orchestrating the game loop and user
- interaction.
- Function:
- `start_game()`: Manages the main game loop, user input, and calls other modules.

2.4.7 Utils Module:

- Description: Provides utility functions and data structures used across other modules.
- Functions/Data:
- `get_level()`: Calculates the player's level.
- `item_emojis`: A dictionary mapping item names to emojis for display.

3. Review of Literature**3.1 Review of Literature :- Basic Theory of the Game**

- Role-playing games work on the idea that a player starts with limited abilities and improves over time by playing the game. As the player performs actions like

Epic RPG

fighting enemies or collecting items, they gain experience, level up, and become stronger.

- Most role-playing games (RPGs) use concepts like health, experience points, inventory, and equipment to measure the progress of players. This project follows the same basic idea but retains the text-based design to ensure easy understanding and learning of the game logic.

3.2 Analysis of Existing Similar Systems: Zork

- Zork, which is one of the earliest examples of a similar system, is a popular text-based adventure game that was developed in the late 1970s. In this game, the player has to control the character in a fantasy world by entering commands in text format.
- The game was quite immersive and engaging, despite the lack of graphics. The current study takes cues from the tradition of text-based adventure games like Zork, but with the addition of modern role-playing game elements like combat, character development, crafting, and inventory management.

4. Technical Description

4.1 Hardware Requirements

- Processor: Intel / AMD processor or equivalent
- RAM: Minimum 2 GB (4 GB or more recommended)
- Storage: 200–300 MB free disk space to support future game data, save files, logs, and feature expansions
- Display: Terminal or console window
- Input Device: Keyboard

Epic RPG

4.2 Software Requirement

- Windows: Windows 10 or later
- Linux: Modern Linux distributions
- macOS: macOS 11 or later

5. System Design and Development

Epic RPG

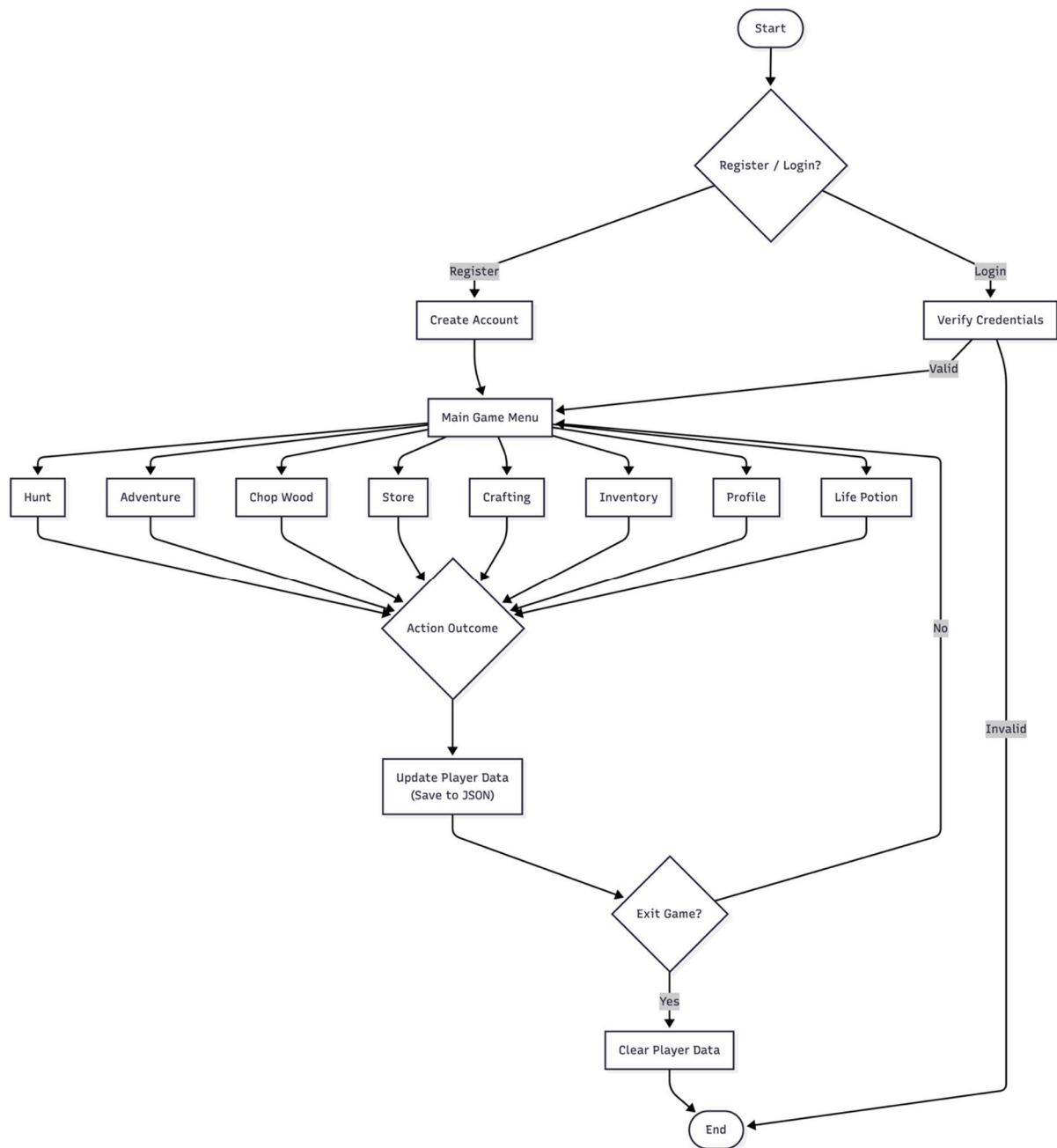


Figure 5.1 Flowchart:

Data flow diagram:

Epic RPG

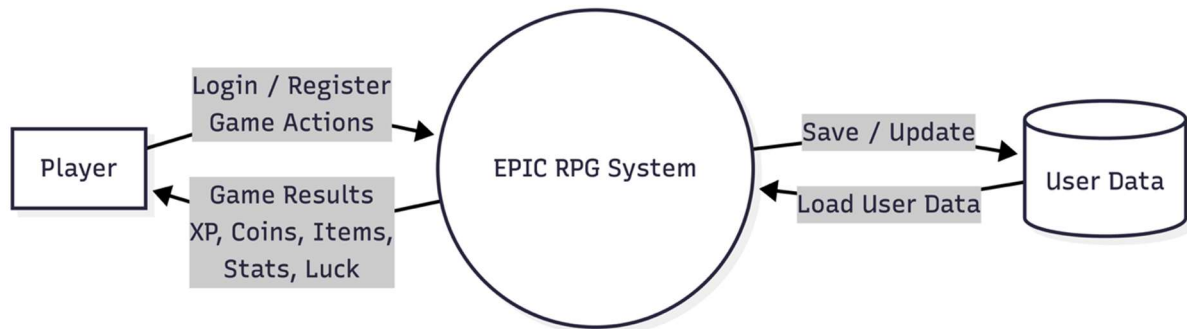


Figure 5.2 DFD-0

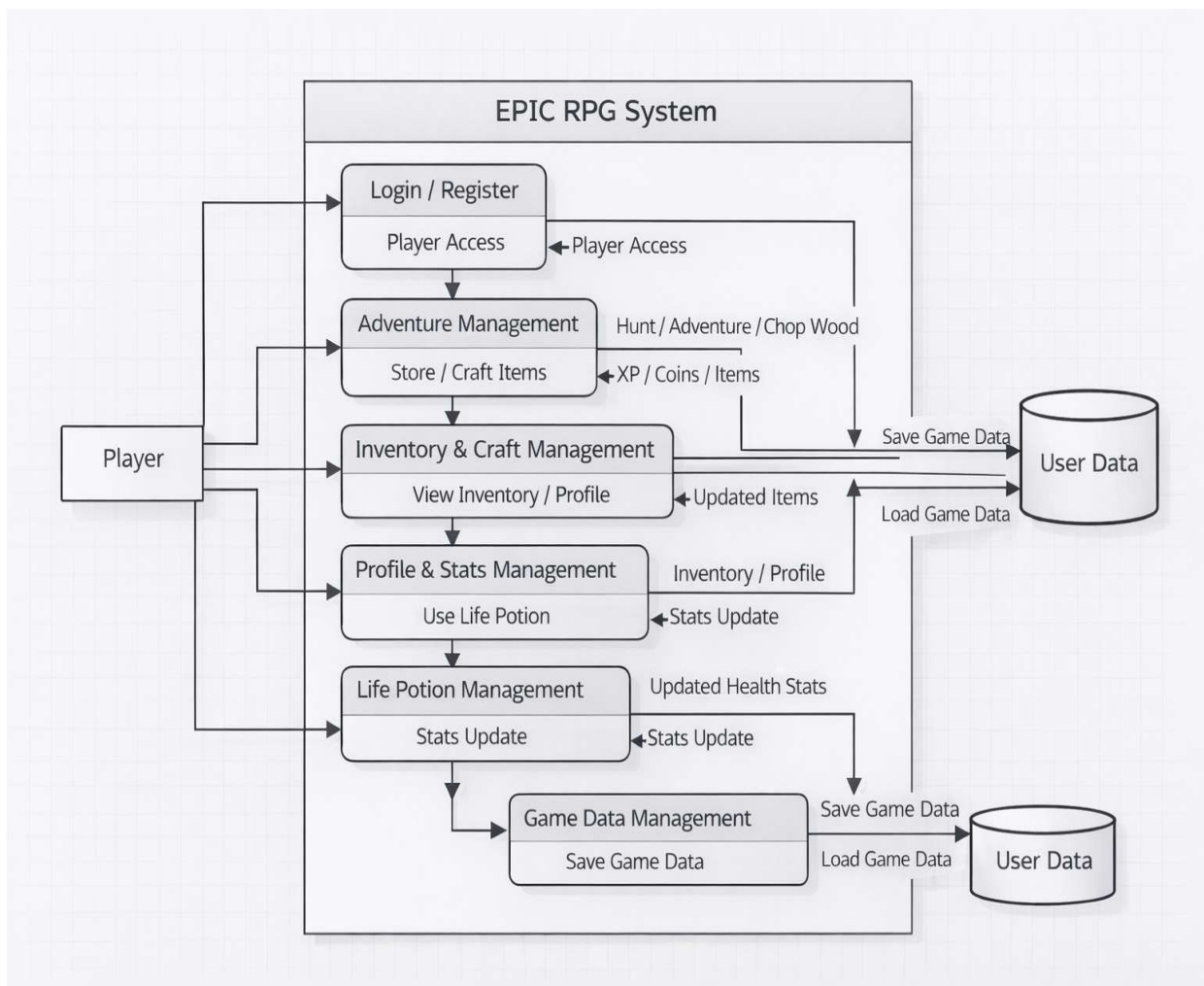


Figure 5.3 DFD Level-1

Epic RPG

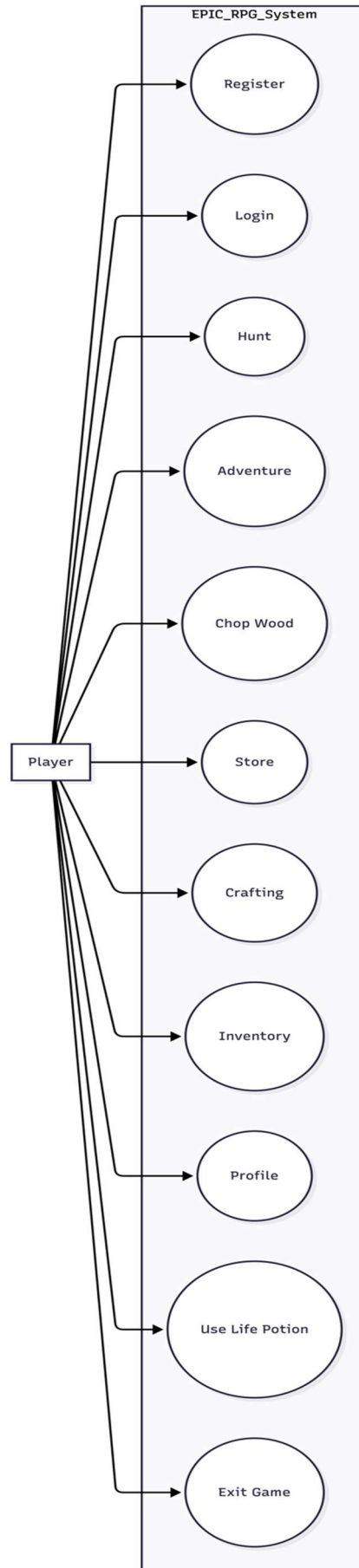


Figure 5.4 Use case diagram

Epic RPG

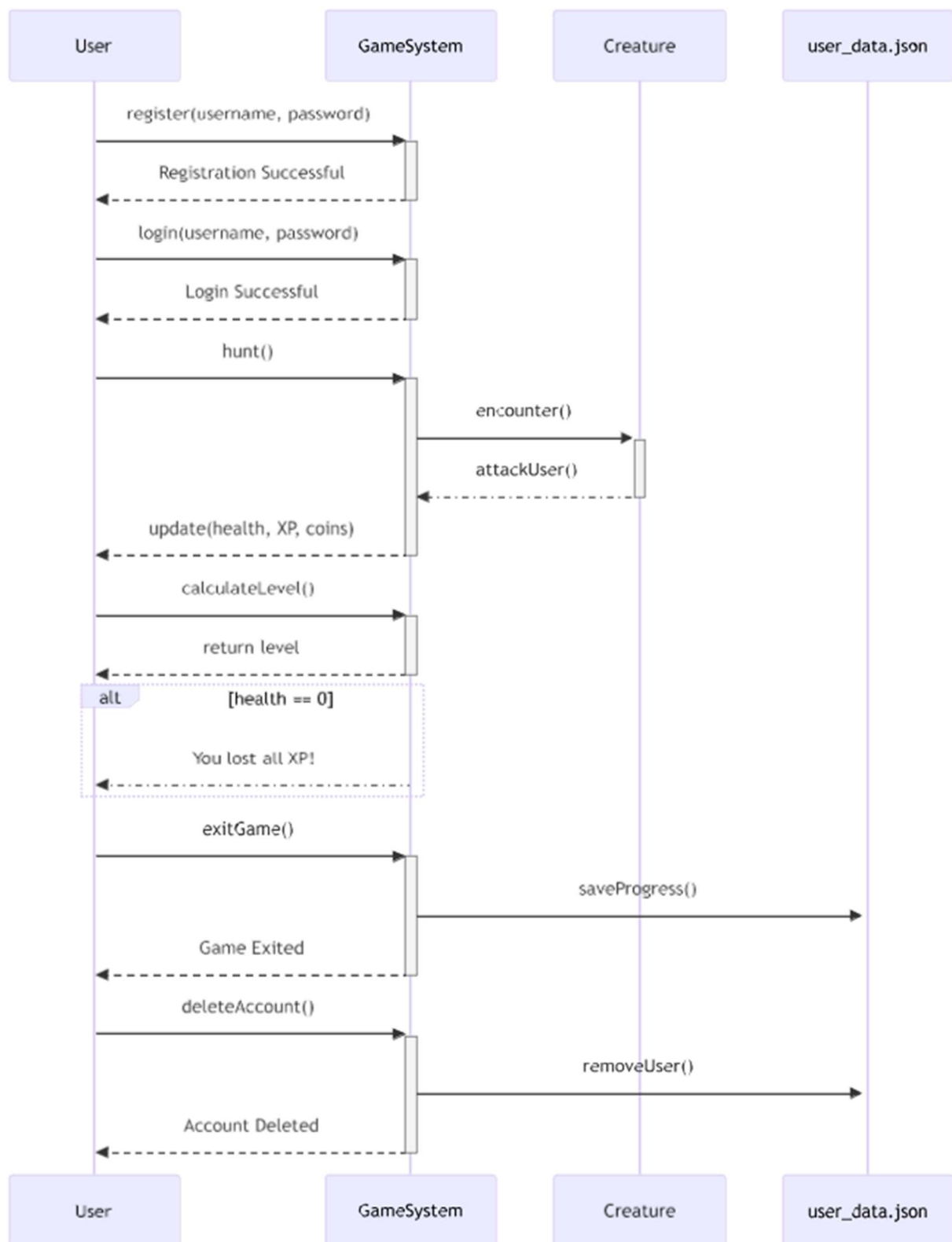


Figure 5.5 Sequential diagram

Epic RPG

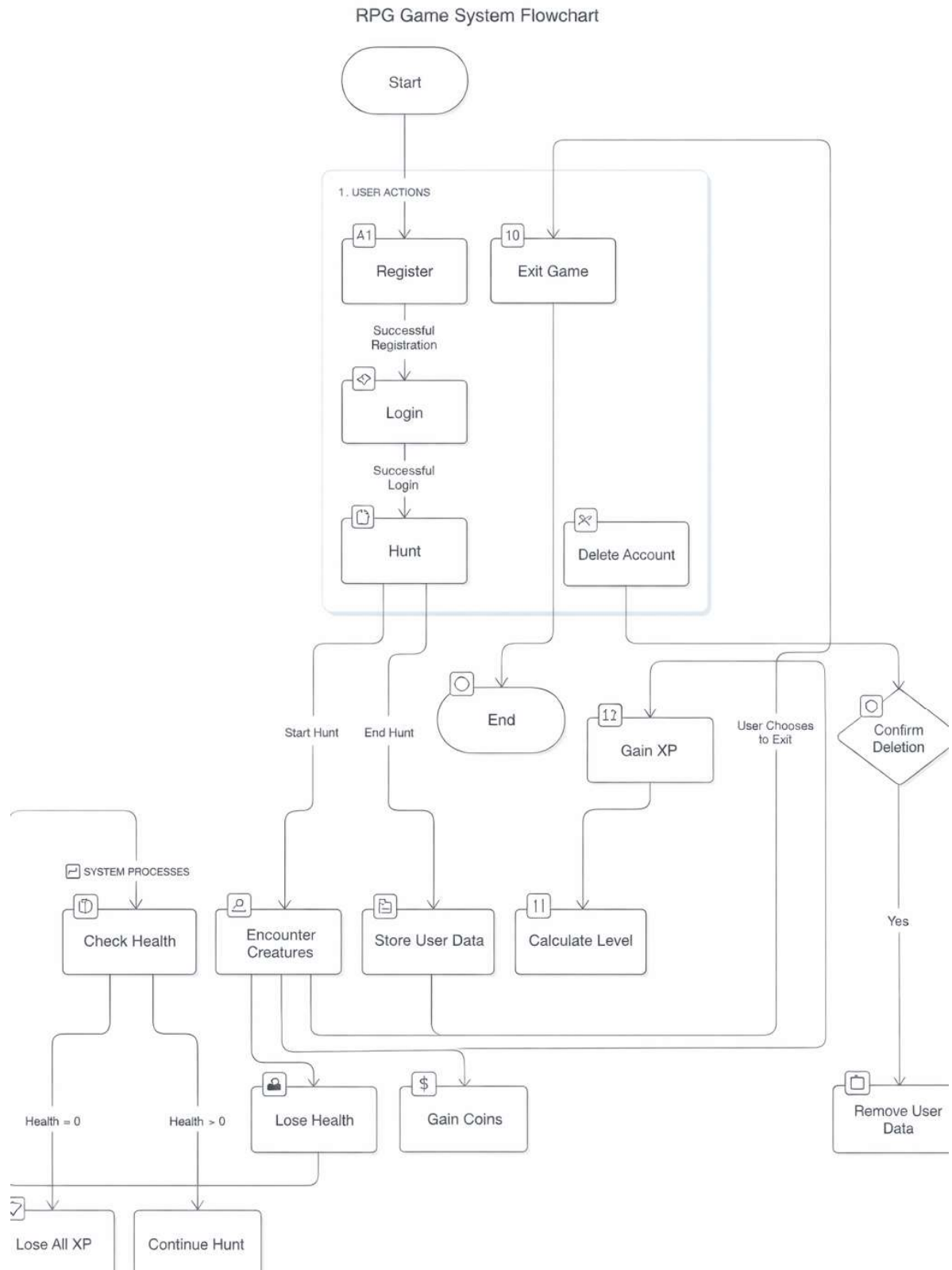


Figure 5.6 Activity diagram:

Epic RPG

5.1 Menu Design

- The menu is the main place where the player controls the game. It shows all the available actions in a simple numbered list so the player can easily decide what to do next.
- The player just enters the number of the option they want, and the game performs that action. After the action is completed, the menu is shown again so the player can continue playing without any confusion.

MENU

1.  **Hunt**
2.  **Adventure**
3.  **Heal using Life Potion**
4.  **View Inventory**
5.  **View Profile**
6.  **Store**
7.  **Chop Wood**
8.  **Reward**
9.  **Craft**
10.  **Delete User**
11.  **Exit**

Epic RPG

5.2 Screen Design

```
adil@192 RPG % python3 main.py

Menu:
1. Register
2. Login
Enter your choice: 2
Enter your username: a
Enter your password: a
Login successful! Welcome, a!

Menu:
1. 🎯 Hunt
2. 🗺️ Adventure
3. 🥬 Heal using Life Potion
4. 🎒 View Inventory
5. 👤 View Profile
6. 🏪 Store
7. 🪵 Chop Wood
8. 🎁 Reward
9. 🛠️ Craft
10. 🗑️ Delete User
11. 🚪 Exit
Enter your choice: █
```

Figure 5.7 Login/Register/Main Menu

Epic RPG

Hunt:

- During gameplay, the player hunts a deer enemy. The system updates health, XP, coins, and inventory in real time. Successful encounters contribute to player progression.

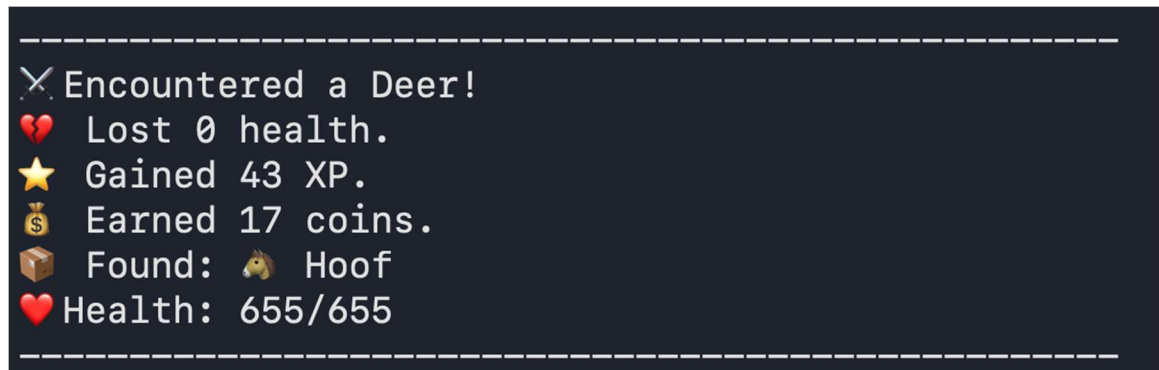


Figure 5.8 Hunt

Adventure:

- The player fights a Demonic Hydra, takes damage, and defeats the enemy. Rewards include XP, coins, and a level increase.

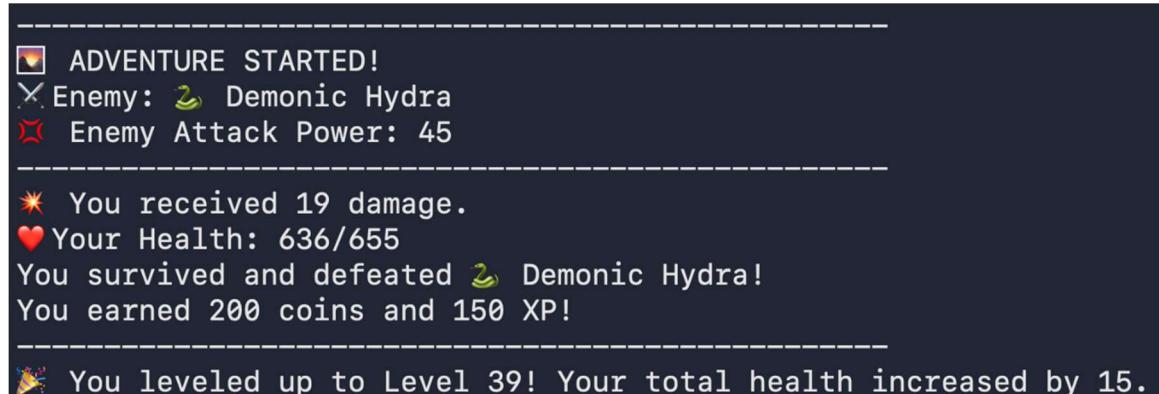


Figure 5.9 Adventure

Epic RPG

Inventory:

- The inventory module maintains a list of all items obtained from battles and exploration. Each item is stored with quantity values and used for healing, crafting, or combat enhancement.

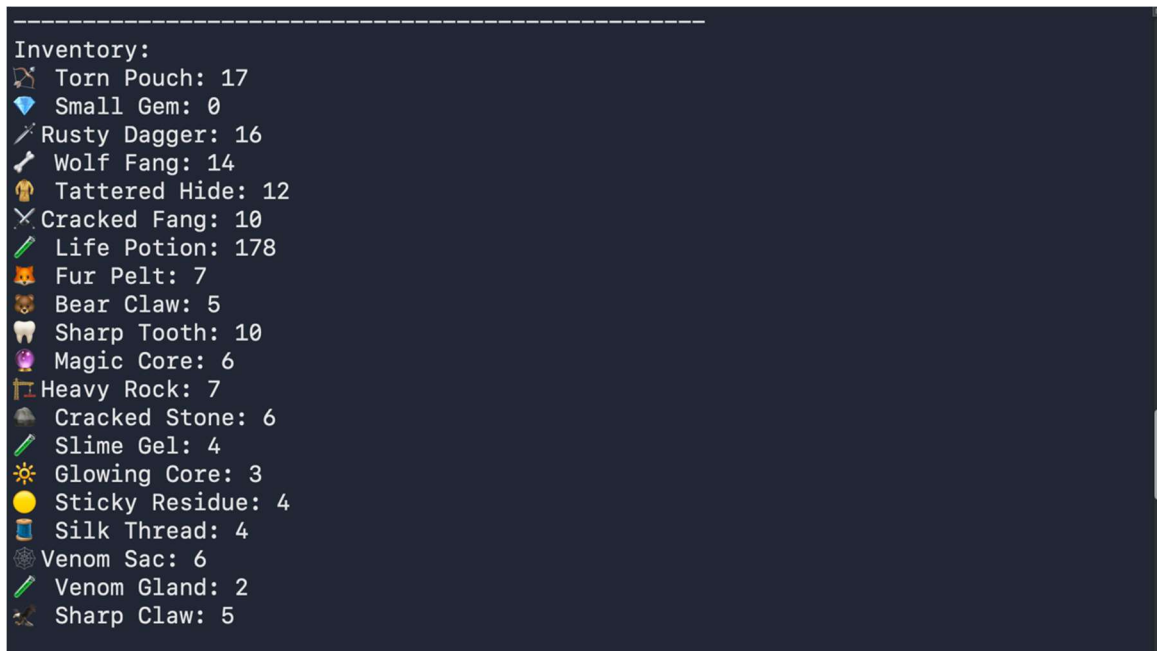


Figure 5.10 Inventory

Profile:

- The player profile module maintains the current character attributes, such as fighting stats, equipment, and resources. All the values are dynamically updated based on the actions performed during gameplay.

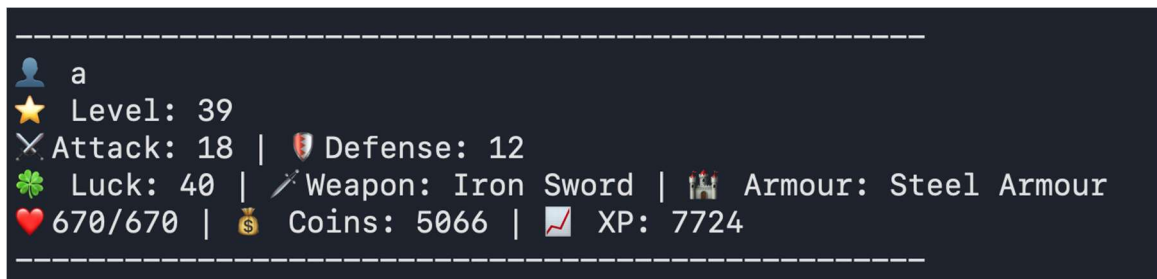


Figure 5.11 Profile

Epic RPG

Store:

- The player has the capability to buy weapons, armor, and items (life potions) from the in-game store. The subtraction of coins is based on the price of the chosen item.

```
Store:
💰 Coins: 5066
1. 🪓🗡️ Wooden Sword: 100 coins
2. 🗡️🔗 Iron Sword: 300 coins
3. 🛡️ Basic Armour: 200 coins
4. 🛡️🔗 Steel Armour: 400 coins
5. 🧴 Life Potion: 20 coins
```

Figure 5.12 Store**Chop:**

- The player gathers resources by interacting with the environment, and the gathered items are then added to the inventory.

```
🪓 You chopped down a tree and obtained: 🌲 Pine Log
```

Figure 5.13 Chop**Reward:**

- The reward system shows the distribution of loot after the successful execution of an action.

```
🎁 You received a reward!
💰 Coins: 396
🧴 Life Potions: 6
```

Figure 5.14 Reward**Craft:**

- The player assembles new items using resources, and each item needs certain resources.

```
Crafting:
1. Wooden Sword (needs 5 🪓 Oak Logs)
2. Lucky Charm (needs 3 💎 Small Gems)
3. Exit
```

Figure 5.15 Craft

6. System Testing

6.1 Testing and Implementation

- The testing was done iteratively throughout the development of the project by running the game in the terminal after the implementation of new features. After the implementation of each feature, the game was run to ensure that it was functioning as expected.
- Whenever there were errors like crashes, incorrect results, or logical errors, the code was analyzed and corrected immediately. This process of continuous testing helped ensure that the game remained stable during the development of new functionalities.

6.2 Testing Methodology

- The testing done was manual testing. Since the game is only running within the command-line interface, the testing was done by choosing different menu options and seeing how the system reacts..
- Various scenarios were considered, such as normal gameplay, invalid input, missing inventory items, cooldowns, and login failures. This evaluation was conducted to ensure that the game handles user input correctly and functions properly during gameplay.

6.3 Test Case Design

- The test cases were designed based on the options available in the game menu. Each test case was designed to test a single feature, and there was close observation of how the player information, such as health, experience points, items in the inventory, and overall statistics, was being updated.

Test Case ID	Menu Option	Description	Expected Result
TC-01	Register	Create a new user account	User account created successfully
TC-02	Login	Login with valid credentials	User logged in
TC-03	Login	Login with invalid credentials	Access denied
TC-04	Hunt	Perform hunt action	XP, coins, and items updated
TC-05	Adventure	Perform adventure	Health reduced or rewards gained
TC-06	Chop Wood	Chop wood	Wood logs added to inventory
TC-07	Store	Buy weapon or item	Inventory updated, stats affected
TC-08	Craft	Craft Wooden Sword	Weapon crafted, attack increased
TC-09	Craft	Craft Lucky Charm	Luck increased, cooldown applied
TC-10	Heal	Use Life Potion	Health restored
TC-11	Inventory	View inventory	Items displayed correctly
TC-12	Profile	View profile	Player stats displayed
TC-13	Reward	Claim reward	XP or coins added
TC-14	Delete User	Delete account	User data removed
TC-15	Exit	Exit game	Program terminates safely

Table No. 6.1

6.4 System Implementation Strategy

- The system was developed in a step-by-step approach to ensure that the process of development remains simple. Basic functionalities such as user registration, login, and menu navigation were developed first.
- After the basic functionalities were developed, gameplay functionalities such as hunting, adventure, inventory management, and profile viewing were developed.
- Features such as the store, crafting system, cooldowns, and the luck attribute were added later. Each feature added to the system was tested using the terminal to see if it was working as expected.
- The process of implementing the features made it easier to detect bugs and correct them while ensuring the system was stable.

7. Conclusion

- This project assisted in learning how a role-playing game functions at a fundamental level with the use of Python. Through the development of a text-based RPG, fundamental concepts in programming, such as functions, data manipulation, file storage, and flow of logic, were learned and implemented.
- The implementation of features such as combat, inventory, crafting, and character development was done in a manner that ensured the system remained simple and easy to understand.
- Through this project, problem-solving skills were also enhanced, as there was a lot of error correction that occurred during the development process. This project has therefore achieved its aim of developing a simple but functional RPG system while offering a good learning experience in Python programming.

8. Learning During Project Work

- Working on this project has been a learning experience. It has helped in understanding how to divide a large problem into smaller parts and how to implement them step by step.
- Through this project, practical knowledge of Python programming was gained, especially in areas like file handling, functions, dictionaries, and program flow control.
- The project has also helped in improving the problem-solving skills of the person. Many errors and logical problems were encountered during the development process of the project, and they were solved by testing and debugging.
- Overall, this project helped build confidence in programming and gave a better understanding of how real applications are developed.